ChatGPT said:

For your **AI Course-II AIML Practical Lab – Category 5**, I'll give you Python programs for all four heuristic search techniques.

I'll keep the programs:

- Simple and beginner-friendly.
- Easy to write in your practical record.
- With sample input and output.
- With comments explaining important steps.

Category 5 – Heuristic Search Techniques

No.	Algorithm	Problem used
1	Greedy Best First Search	8 Puzzle
2	A* Search	8 Puzzle
3	AO* Search	AND-OR Graph
4	Memory Bounded Heuristic Search (IDA*)	Missionaries and Cannibals

---

1. Greedy Best First Search – 8 Puzzle Problem

Aim: To implement Greedy Best First Search using a heuristic function to solve the 8 Puzzle problem.

Python Program

Python

```
from heapq import heappush, heappop

goal = (1, 2, 3,
        4, 5, 6,
        7, 8, 0)

def heuristic(state):
    distance = 0

    for i in range(9):
        if state[i] != 0:
            goal_pos = state[i] - 1
            distance += abs(i // 3 - goal_pos // 3)
            distance += abs(i % 3 - goal_pos % 3)

    return distance


def greedy_bfs(start):
    pq = []
    heappush(pq, (heuristic(start), start))

    visited = set()

    while pq:
        h, state = heappop(pq)

        if state == goal:
            print("Goal Reached!")
```

Output

Plain text

```
Goal Reached!
(1, 2, 3, 4, 5, 6, 7, 8, 0)
```

Explanation:

1. Calculate the heuristic value using Manhattan distance.
2. Insert the initial state into a priority queue.
3. Select the state with the lowest heuristic value.
4. Generate possible movements of the empty tile.
5. Repeat until the goal state is reached.

2. A* Search – 8 Puzzle Problem

Aim: To implement A* Search using the evaluation function.

$$f(n) = g(n) + h(n)$$

Where:

- `g(n)` = Cost from initial state to current state.
- `h(n)` = Estimated cost from current state to goal.
- `f(n)` = Total estimated cost.

Python Program

Python

```python
from heapq import heappush, heappop

goal = (1, 2, 3,
        4, 5, 6,
        7, 8, 0)

def heuristic(state):
    distance = 0

    for i in range(9):
        if state[i] != 0:
            target = state[i] - 1

            distance += abs(i // 3 - target // 3)
            distance += abs(i % 3 - target % 3)

    return distance


def astar(start):
    pq = []
    heappush(pq, (heuristic(start), 0, start))

    visited = {}

    while pq:
        f, g, state = heappop(pq)

        if state == goal:
```

Output

Plain text

```
Goal Reached!
Total Cost: 2
(1, 2, 3, 4, 5, 6, 7, 8, 0)
```

Explanation:

1. Calculate `g(n)` and `h(n)`.
2. Calculate `f(n) = g(n) + h(n)`.

3. Select the state with the lowest f-value.
4. Generate the next possible states.
5. Continue until the goal state is reached.

3. AO* Search – AND-OR Graph

Aim: To implement AO* Search for solving an AND-OR graph.

Python Program

Python

```
    'D': [['H', 'I']],
    'E': [],
    'F': [],
    'G': [],
    'H': [],
    'I': []
}

heuristic = {
    'A': 10,
    'B': 4,
    'C': 2,
    'D': 5,
    'E': 0,
    'F': 0,
    'G': 0,
    'H': 0,
    'I': 0
}

solution = {}

def ao_star(node):
```

Output

Plain text

```
Minimum Cost: 0
Solution Path:
A -> D -> H -> I ->
```

Explanation:

1. Start from the initial node A.
2. Identify the available OR alternatives.
3. For an AND branch, calculate the sum of all child costs.
4. Select the branch with the minimum cost.
5. Update the heuristic value.
6. Continue until terminal nodes are reached.

Important concept:

- OR node: Select one best child.
- AND node: All selected children must be solved.

4. Memory Bounded Heuristic Search – Missionaries and Cannibals

Aim: To implement a memory-bounded heuristic search algorithm using IDA* for the Missionaries and Cannibals problem.

IDA* (Iterative Deepening A*) combines A* evaluation with iterative deepening to reduce memory usage.

Python Program

Python

```
# Missionaries and Cannibals using IDA*

def heuristic(state):
    m, c, boat = state
    return (m + c + 1) // 2


def is_valid(m, c):
    if m < 0 or c < 0 or m > 3 or c > 3:
        return False

    if m > 0 and c > m:
        return False

    mr = 3 - m
    cr = 3 - c

    if mr > 0 and cr > mr:
        return False

    return True


def successors(state):
    m, c, boat = state
    moves = [(1, 0), (2, 0), (0, 1),
             (0, 2), (1, 1)]

    result = []
```

Sample Output

The program prints the sequence of states from the initial state to the goal state.

Plain text

```
Solution Found:
(3, 3, 0)
(3, 1, 1)
(3, 2, 0)
...
(0, 0, 1)
```

Explanation:

1. Start with the initial state (3,3,0).
2. Calculate the heuristic value.
3. Calculate `f(n) = g(n) + h(n)`.
4. If the f-value exceeds the current limit, reject that path.
5. Increase the limit and repeat the search.
6. Stop when the goal state (0,0,1) is reached.

Important viva questions

Question	Answer
What is heuristic search?	Search using knowledge to estimate the distance to the goal.
What is Greedy Best First Search?	It selects the node with the lowest h(n).

What is A* Search?	It uses f(n) = g(n) + h(n).
What is AO* Search?	It searches AND-OR graphs.
What is an AND node?	All child nodes must be solved.
What is an OR node?	Any one suitable child can be selected.
What is IDA*?	Iterative Deepening A* Search.
Why is IDA* memory efficient?	It uses depth-first search and stores only the current path.
Which algorithm uses f(n)?	A* and IDA*.
Which algorithm uses only h(n)?	Greedy Best First Search.

Comparison of all four algorithms

Algorithm	Evaluation	Main advantage
Greedy BFS	h(n)	Fast goal-directed search
A*	g(n) + h(n)	Optimal with an admissible heuristic
AO*	AND-OR cost	Solves problem decomposition
IDA*	g(n) + h(n) with thresholds	Low memory usage

One important note for your practical: IDA* is a memory-bounded heuristic search technique, but it is not SMA* (Simplified Memory-Bounded A*). If your faculty specifically expects SMA*, use that name and implementation instead.

I can also teach you these four programs one at a time, explaining every Python statement so you can write them independently in your lab exam.